Introduction to Algorithms

Email: grad.saeed@gmail.com

Github: github.com/saeedgrad

Divide-and-Conquer

- The divide-and-conquer method is a powerful strategy for designing asymptotically efficient algorithms.
- We saw an example of divide-and-conquer in when learning about merge sort.
- We'll explore applications of the divide-and-conquer method
- We'll explore mathematical tools to solve the recurrences that arise when analyzing divide-and-conquer algorithms.

Divide-and-Conquer

- Recall that for divide-and-conquer, you solve a given problem (instance) recursively.
- If the problem is small enough-the base case-you just solve it directly without recursing.
- Otherwise-the recursive case-you perform three characteristic steps:
- Divide
- Conquer
- Combine

Divide-and-Conquer

- Divide
  Divide the problem into one or more subproblems that are smaller instances of the same problem.

- Conquer
  Conquer the subproblems by solving them recursively.

- Combine
  Combine the subproblem solutions to form a solution to the original problem.

Recurrences

- To analyze recursive divide-and-conquer algorithms, we'll need some mathematical tools.
- A recurrence is an equation that describes a function in terms of its value on other, typically smaller, arguments.
- The general form of a recurrence is an equation or inequality that describes a function over the integers or reals using the function itself.

Recurrences

- A recurrence contains two or more cases, depending on the argument. If a case involves the recursive invocation of the function on different (usually smaller) inputs, it is a recursive case.
- If a case does not involve a recursive invocation, it is a base case.
- The recurrence is well defined if there is at least one function that satisfies it, and ill defined otherwise.

Algorithmic recurrences

We'll be particularly interested in recurrences that describe the running times of divide-and-conquer algorithms. A recurrence $T(n)$ is ***algorithmic*** if, for every sufficiently large ***threshold*** constant $n_0 > 0$, the following two properties hold:

1. For all $n < n_0$, we have $T(n) = \Theta(1)$.
2. For all $n \geq n_0$, every path of recursion terminates in a defined base case within a finite number of recursive invocations.

Examples of recurrences

$$T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ T(n-1) + 1 & \text{if } n > 1 . \end{cases}$$

$$T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ 2T(n/2) + n & \text{if } n \geq 1 . \end{cases}$$

$$T(n) = \begin{cases} 0 & \text{if } n = 2 , \\ T(\sqrt{n}) + 1 & \text{if } n > 2 . \end{cases}$$

$$T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ T(n/3) + T(2n/3) + n & \text{if } n > 1 . \end{cases}$$

Solving recurrences

1- Substitution:

In the ***substitution method*** (Section 4.3), you guess the form of a bound and then use mathematical induction to prove your guess correct and solve for constants. This method is perhaps the most robust method for solving recurrences, but it also requires you to make a good guess and to produce an inductive proof.

Solving recurrences

2- Recursion-tree method:

The ***recursion-tree method*** (Section 4.4) models the recurrence as a tree whose nodes represent the costs incurred at various levels of the recursion. To solve the recurrence, you determine the costs at each level and add them up, perhaps using techniques for bounding summations from Section A.2. Even if you don't use this method to formally prove a bound, it can be helpful in guessing the form of the bound for use in the substitution method.

Solving recurrences

3- Master theorem:

The ***master method*** (Sections 4.5 and 4.6) is the easiest method, when it applies. It provides bounds for recurrences of the form

$$T(n) = aT(n/b) + f(n) ,$$

where $a > 0$ and $b > 1$ are constants and $f(n)$ is a given "driving" function. This type of recurrence tends to arise more frequently in the study of algorithms than any other. It characterizes a divide-and-conquer algorithm that creates a subproblems, each of which is $1/b$ times the size of the original problem, using $f(n)$ time for the divide and combine steps. To apply the master method, you need to memorize three cases, but once you do, you can easily determine asymptotic bounds on running times for many divide-and-conquer algorithms.

Solving recurrences

4- Generating functions:

Define the ***generating function*** (or ***formal power series***) $\mathcal{F}$ as

$$\mathcal{F}(z) = \sum_{i=0}^{\infty} F_i z^i$$
$$= 0 + z + z^2 + 2z^3 + 3z^4 + 5z^5 + 8z^6 + 13z^7 + 21z^8 + \cdots ,$$

where F_i is the ith Fibonacci number.